Triple-Packed Posit MAC: Low-Bitwidth Inference via Tri-Product DSP48E2 Overpacking for CNNs

Nishith Akula, Sujit Ghantasala ,Komaragiri Sai Vishwanath Rohit, Rahul Mukundhan, Madhav Rao
IIIT Bangalore, {a.nishith,sujit.ghantasala,komaragiri.sai,rahul.mukundhan,mr}@iiitb.ac.in

2026 IEEE International Symposium on Circuits and Systems (ISCAS) | 979-8-3315-7769-8/26/$31.00 ©2026 IEEE | DOI: 10.1109/ISCAS66217.2026.11562313

***Abstract*—We present *Triple-Packed Posit MAC*, an FPGA accelerator that packs Xilinx UltraScale+ DSP48E2 blocks to execute three exact 4-bit posit multiplications per cycle, tripling throughput while using only one DSP. The design supports `Posit(8,1)`, `Posit(8,2)`, and `Posit(8,3)` with just 14 LUTs for carry correction. Integrated into a matrix-multiplication and convolution pipeline on a ZCU104 board, it targets LeNet-5 inference on MNIST. Compared to a baseline posit engine, this approach reduces LUT usage by 25%, frees two DSPs per MAC, and increases convolution-layer MAC throughput by 3× with identical accuracy. RMSE analysis shows `Posit(8,1)` is within 0.02 of FP16 for 3-MAC operations, and layer-level RMSE remains under 0.5 for both Conv2D and Dense layers. In MNIST inference, `Posit(8,1)` achieves FP32 Top-1 accuracy within 0.3% (90.5% vs. 90.8%) and 99% Top-5 accuracy. This demonstrates up to 3× higher compute density and 15–25% logic savings without compromising numerical fidelity.**

***Index Terms*—POSIT, MAC, DSP packing, CNN, matrix multiplication, im2col**

I. Introduction

Convolution in deep neural networks is often reshaped into matrix multiplication (e.g., via *im2col* [25]). On FPGAs, high-throughput MACs rely on DSP48E2 blocks [29] with a 27×18-bit signed multiplier and 48-bit accumulator. While ideal for high precision, these blocks are frequently under-utilized with low-bitwidth data. DSP packing schemes combine several small-width multiplications per DSP [22], [24], but most INT4 methods still yield only two useful products per cycle due to overlapping result regions, operand reuse, or shared input patterns that limit independent outputs [1]–[3].

To overcome this bottleneck, we propose a recursive-multiplication solution. Exploiting the DSP48E2's 27-bit A, 18-bit B, and 48-bit C inputs [23], our *Triple-Packed Posit MAC* computes three useful 4-bit multiplications per cycle. By efficiently combining partial products within the native multiplier–accumulator, the approach is well-suited to the mantissa multiplication of 8-bit posits. We target `posit(8,1)`, `posit(8,2)`, and `posit(8,3)`, where mantissas are $\leq$4 bits, enabling three exact posit multiplications per DSP48E2 each cycle without approximation, thereby maximizing DSP utilization and reducing LUT usage.

We implement a customized posit MAC performing three multiplications and two additions in a single DSP48E2, integrate it into a matrix-multiplication/convolution accelerator, and deploy it on a Xilinx UltraScale+ FPGA. Evaluating all LeNet-5 layers shows that 8-bit posit inference with the proposed *Triple-Packed Posit MAC* achieves accuracy comparable to FP32 and FP16, reinforcing that low-bit posits can rival or surpass floating-point in inference. This aligns with prior evidence: 8-bit posit networks matching FP32 [15], high-accuracy CNN inference using 8-bit fixed-point posits [12], and recent FPGA-oriented implementations [7], [8], [13].

II. Preliminaries and Related Work

The posit format, proposed as a compact alternative to IEEE 754, provides higher precision and dynamic range at the same bit width [16], [17]. It consists of a sign, variable-length regime, exponent, and fraction fields (Figure 1). The MSB is the sign; the regime is a run of identical bits ending with the opposite bit, encoding $r = -m$ for m leading 0s or $r = m-1$ for 1s. The following exponent (es) and fraction fields balance range and precision—smaller es yields finer precision, larger es broadens range, with useed $= 2^{2^{es}}$. For 8-bit posits with $es \in \{1, 2, 3\}$, decoding requires a Leading Zero/One Detector (LZD/LOD) to locate the regime boundary before parsing exponent and fraction. This adaptive encoding supports efficient arithmetic [5], enabling FPGA accelerators [4], [7]. Toolchains include hardware generators [9], [10], HW/SW libraries [18], [19], and simulation packages such as `sgpositpy` [28].

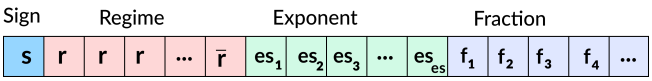

Fig. 1. Posit representation.

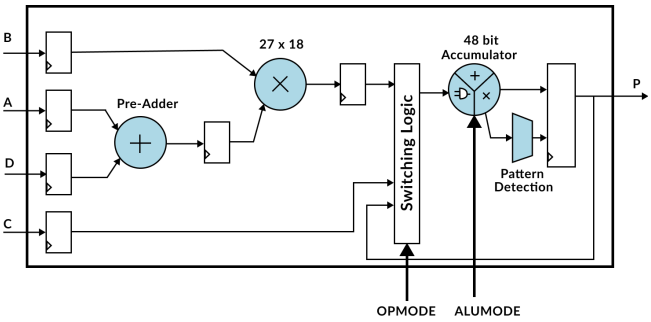

Fig. 2. Schematic of DSP48E2 Architecture.

We implement 4-bit multiplication by recursively splitting each operand into a high 3-bit segment and a low 1-bit segment, then expanding the product accordingly.

$$X = x_3x_2x_1x_0, \quad Y = y_3y_2y_1y_0$$
$$X \cdot Y = 2^2(x_3x_2x_1)(y_3y_2y_1) + 2^2x_0(y_3y_2) + 2^2y_0(x_3x_2) + 2(x_0y_1) + 2(y_0x_1) + x_0y_0 \quad (1)$$

The 3+1 split is selected to efficiently map the dominant 3-bit $\times$ 3-bit partial product onto the DSP48E2 (Figure 2) implements a signed multiply–accumulate operation with input ports (27-bit A, 18-bit B) while enabling spatial separation of multiple packed results. A symmetric 2+2 split generates partial products of comparable magnitude, which require larger alignment shifts. By contrast, the 3+1 partition confines the remaining low-order terms to lightweight corrections, allowing three independent multiplications to be packed per DSP without violating bit-width constraints.

Using this divide-and-conquer scheme [26], [27] lets us pack three multiplications per DSP, thereby maximizing utilization.

III. Proposed Methodology - Triple-Packed Posit MAC

A. INT4 Packing in DSP Blocks

Modern DSP blocks in hardware like FPGAs can perform large multiplications efficiently. However, when using low-bitwidth numbers like 4-bit integers (INT4), these DSPs are often underutilized. To make better use of their capacity, we pack multiple INT4 values into the DSP inputs so that several small multiplications are performed at once. Our packing strategy is illustrated in Figure 3. In our method, we combine three 4-bit weights and three 4-bit activations into a single packed format. This allows the DSP block to compute three separate INT4 $\times$ INT4 products in parallel, which is explained below. Let three 4-bit weights be:

$$W_i = w_{i,3}w_{i,2}w_{i,1}w_{i,0}, \quad i = 1, 2, 3, \quad (2)$$

and three 4-bit activations:

$$A_j = a_{j,3}a_{j,2}a_{j,1}a_{j,0}, \quad j = 1, 2, 3. \quad (3)$$

Define the MSBs (most significant 3 bits) as:

$$w_i = (w_{i,3}w_{i,2}w_{i,1}), \quad a_j = (a_{j,3}a_{j,2}a_{j,1}). \quad (4)$$

We pack them into DSP inputs A and B as:

$$A = w_1 + w_2 \cdot 2^{12} + w_3 \cdot 2^{23}, \qquad B = a_1 + a_2 \cdot 2^6 + a_3 \cdot 2^{12}. \quad (5)$$

The packed multiplication yields:

$$A \cdot B = w_1a_1 + w_1a_2 \cdot 2^6 + w_1a_3 \cdot 2^{12} + w_2a_1 \cdot 2^{12} + w_2a_2 \cdot 2^{18} + w_2a_3 \cdot 2^{24} + w_3a_1 \cdot 2^{23} + w_3a_2 \cdot 2^{29} + w_3a_3 \cdot 2^{35}. \quad (6)$$

Among the resulting terms in the packed multiplication, three products including w_1a_1, w_2a_2, and w_3a_3—are positioned to occupy distinct bit ranges in the final result. Specifically, w_1a_1 lies within bits [5:0], w_2a_2 is mapped to bits [23:18], and w_3a_3 is located at bits [40:35]. These regions are carefully selected to allow each product to be extracted independently for use in MAC operations.

B. Overlap and Error Conditions

Packing three independent 3-bit multiplications into a single DSP requires careful placement of partial products to avoid bit overlap. However, this separation is not perfect—some terms still interfere or cause carry issues, leading to ambiguities in result extraction. To address these, we introduce correction methods. We address these in two main scenarios:

1) **Bit 23 Overlap:** The partial product $w_2a_2 \cdot 2^{18}$ spans bits [23:18], while $w_3a_1 \cdot 2^{23}$ also writes to bit 23, causing interference. This overlap makes it difficult to extract the MSB of w_2a_2, which is needed to reconstruct the second output P_2. To resolve this, we use an empirical rule: if $(W_2 + A_2 > 16)$ or $(W_2 = 8$ and $A_2 = 8)$, where W_2 and A_2 are from Equations (2) and (3), the MSB is inferred as 1; otherwise, it is 0. We extract only bits [22:18] and concatenate the predicted MSB, recovering the full 6-bit result.
2) **Carry into Bit 18:** The terms $w_1a_3 \cdot 2^{12}$ and $w_2a_1 \cdot 2^{12}$ share the same bit range, and their sum may exceed 6 bits. If $w_1a_3 + w_2a_1 > 63$, a carry propagates into bit 18, overlapping with the start of w_2a_2 and corrupting the second product. To prevent this, we check the overflow condition and subtract 2^{18} via the DSP's additive input C when needed. This correction preserves bit 18 and enables accurate extraction of w_2a_2. The full error logic uses only 14 LUTs.

C. Packing into the DSP Additive Input C

In the recursive multiplication expansion (Equation 1), several lower-order partial products — specifically x_0y_1, y_0x_1, and x_0y_0, contribute to the final result with a weight of 2^2. Due to packing limitations in the DSP's multiplicative inputs A and B, these terms cannot be directly included there. To account for their contribution, we compute a correction term s_i for each 4-bit input pair (w_i, a_i) as expressed in Equation 7, where $w_{i,0}$ and $a_{i,0}$ are the least significant bits (LSBs), and $w_{i,1}$, $a_{i,1}$ are the next higher bits of the respective operands.

$$s_i = 2w_{i,0}a_{i,1} + 2a_{i,0}w_{i,1} + w_{i,0}a_{i,0} \quad (7)$$

Importantly, we do not add the full value of s_i to the DSP. Instead, we extract only its most significant bit (MSB), denoted $b_i = \text{MSB}(s_i)$, since it alone contributes at the 2^2 weight in the product. This bit is aligned with the exponent of the main product at that stage and can be safely accumulated via the DSP's additive input C. Additionally, we incorporate cross-terms such as $w_{i,0}(a_{i,3}a_{i,2})$ and $a_{i,0}(w_{i,3}w_{i,2})$, which also contribute at the 2^2 position and originate from multiplying LSBs with higher-order bits of the other operand. In cases where $w_1a_3 + w_2a_1 > 63$, a correction of 2^{18} is subtracted through the same C input to prevent carry into bit 18. The complete additive input C is computed as shown in Equation 8, with shift offsets $p_1 = 0$, $p_2 = 18$, and $p_3 = 35$ aligning each correction term with its corresponding packed product. The correction is applied conditionally via the $-\delta \cdot 2^{18}$ term.

$$C = \sum_{i=1}^{3} 2^{p_i} \left[w_{i,0}(a_{i,3}a_{i,2}) + a_{i,0}(w_{i,3}w_{i,2}) + b_i\right] - \delta \cdot 2^{18} \quad (8)$$

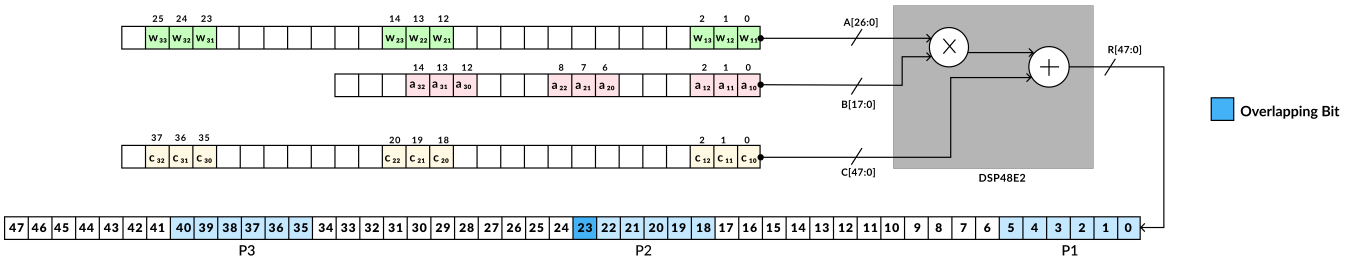

Fig. 3. INT4 packing strategy in DSP blocks.

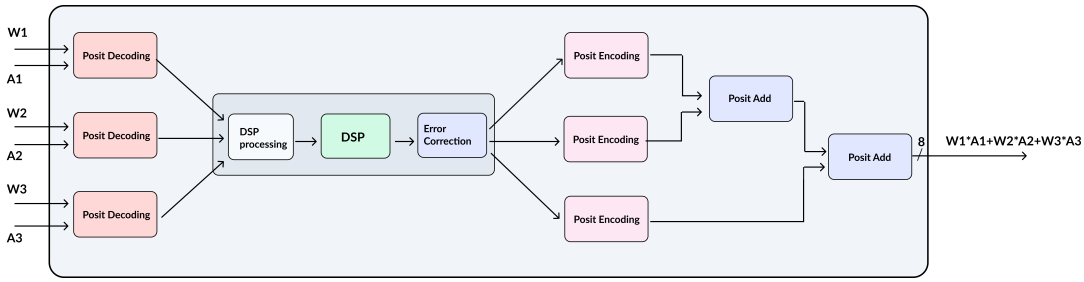

Fig. 4. System Architecture for Triple-Packed Posit MAC Integration

$$\delta = \begin{cases} 1, & \text{if } w_1 a_3 + w_2 a_1 > 63 \\ 0, & \text{otherwise} \end{cases}$$

D. Final Output Recovery

After the DSP computes the packed result $R = A \cdot B + C$, we extract the high-order 6-bit outputs corresponding to each packed product. Specifically, we take bits [5:0] from R to recover P_1, bits [23:18] for P_2, and bits [40:35] for P_3. An overlap affects bit 23 due to adjacent packed multiplications. To address this, we apply a correction to the MSB of $R[23:18]$ for the partial product P_2. The most significant bit (MSB) of $w_2 a_2$ is predicted using the following condition, as part of our approximate logic methodology described earlier.

The predicted MSB is concatenated with the lower 5 bits $R[22:18]$ to reconstruct the full 6-bit result for P_2. Each 6-bit product is then left-shifted by two bits (equivalent to multiplying by 2^2) to align with the original 8-bit scale. Finally, the two least significant bits of s_i, denoted $\text{LSB}_2(s_i)$, are added to complete the result.

$$P_1 = (R[5:0] \ll 2) + \text{LSB}_2(s_1),$$
$$P_2 = (\text{Corrected}(R[23:18]) \ll 2) + \text{LSB}_2(s_2),$$
$$P_3 = (R[40:35] \ll 2) + \text{LSB}_2(s_3).$$

This method enables accurate recovery of three parallel 4-bit multiplications from a single DSP computation. The accurate reconstruction of each product validates the effectiveness of our packing strategy.

E. Integration of Posit Into System Design

We integrate posit operations into our DSP packing framework to improve numerical accuracy without extra hardware. Using 8-bit posits ($N = 8$) with ES $\in \{1, 2, 3\}$, up to 4 mantissa bits are allowed when ES $= 1$, matching the DSP's 4-bit multiplication format. The posit MAC is performed in four stages: pre-processing, DSP packing, post-processing, and addition.

In the first stage, three parallel posit decoders extract and zero-pad mantissas to 4 bits. In the DSP packing stage, only the $m_1 m_2$ product is computed using packed 4-bit integers, and the hidden-bit terms are handled later. The post-processing stage encodes the DSP results back to 8-bit posits, performing composition, normalization, and rounding. Finally, the posit products are added in two serial stages, with pipelines for decoding, alignment, mantissa addition/subtraction, and normalization. This four-stage pipeline efficiently integrates posit arithmetic into a MAC, ensuring high performance, utilization, and accuracy.

IV. Experimental Setup and Results

We evaluated the Triple-Packed Posit MAC architecture on a Xilinx Zynq UltraScale+ MPSoC ZCU104 FPGA using Vivado, targeting three CNN components: a MAC unit, Conv2D layer, and Fully Connected layer, tested with the LeNet-5 model on the MNIST dataset (60,000 training, 10,000 test images). Inference accuracy was simulated using quantized weights from a PyTorch-trained LeNet-5 model. Hardware evaluations focused on resource usage (LUTs, DSPs) and accuracy trade-offs across POSIT formats (8,1), (8,2), and (8,3), compared to IEEE 754 FP16/FP32.

A. Resource Utilization

Table I summarizes the resource utilization (DSP and LUT) of our MAC designs for POSIT configurations (8,1), (8,2), and (8,3), compared to state-of-the-art (SOTA) implementations from [6], [14], and adapted versions of [9]. IEEE FP32

TABLE I
COMPREHENSIVE RESOURCE UTILIZATION COMPARISON

	Proposed Design			[6]			[14]			[9]			**IEEE**	
Config	(8,1)	(8,2)	(8,3)	(8,1)	(8,2)	(8,3)	(8,1)	(8,2)	(8,3)	(8,1)	(8,2)	(8,3)	FP32	FP16
LUTs	618	555	512	700	630	620	775	740	600	926	894	915	1392	912
DSPs	1	1	1	0	0	0	0	0	0	3	3	3	3	0
MAC Type	Exact			Exact			Approximate			Exact			Exact	

and FP16 implementations using the Xilinx Floating-Point IP were also evaluated. The proposed Triple-Packed Posit MAC achieves a 12% LUT reduction over [6] and 20% over [14] for POSIT(8,1), a 12% reduction over [6] and 25% over [14] for POSIT(8,2), and 17.4% and 14.6% fewer LUTs than [6] and [14] for POSIT(8,3), respectively. Importantly, our design maintains exact MAC functionality compared to [14], highlighting its area efficiency while preserving computational precision.

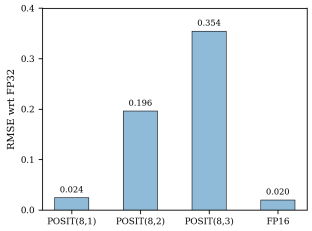

Fig. 5. Comparison of RMSE values with respect to FP32 for POSIT(8,1), POSIT(8,2), POSIT(8,3), and FP16 formats.

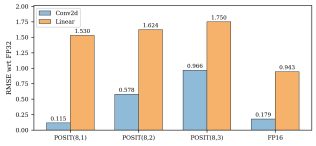

Fig. 6. RMSE comparison for Conv2D layer and Fully Connected layer outputs against FP32 across different number formats: POSIT(8,1), POSIT(8,2), POSIT(8,3), and FP16.

B. RMSE Analysis: 3-MAC Operation

The RMSE for the 3-MAC operation across different POSIT configurations was computed to assess numerical deviations from IEEE FP32. As shown in Figure 5, POSIT(8,1) strikes a balance between precision and efficiency, with RMSE similar to FP16 but lower resource usage. In contrast, POSIT(8,2) and POSIT(8,3) show higher RMSE due to reduced precision from their larger exponent bits.

C. RMSE Analysis: Layer-Level Computations

RMSE was analyzed at the full-layer level for Conv2D and Fully Connected layers to evaluate accumulated numerical errors. As shown in Figure 6, POSIT(8,1) achieves the lowest RMSE (0.115 for Conv2D, 1.530 for Dense), offering strong stability. It is comparable to FP16 in Conv2D (0.179) but shows higher error in Dense (0.943). POSIT(8,2) and POSIT(8,3) exhibit increasing RMSE, with POSIT(8,3) peaking at 1.750 in Dense. These results highlight POSIT(8,1) as the best balance of accuracy and efficiency.

D. Inference Accuracy Analysis

Inference accuracy on the MNIST dataset using LeNet-5 was evaluated with quantized weights and activations in various formats. Table II shows the Top-1 and Top-5 accuracy, FP32 baseline, and prediction match. POSIT(8,1) achieved near-FP32 accuracy with lower hardware cost, while FP16 was almost identical to FP32 due to its higher precision. POSIT(8,2) and POSIT(8,3) had slightly lower Top-1 accuracy but maintained high Top-5 accuracy, with POSIT(8,2) achieving a 96.9% match to FP32 predictions. The source code and hardware designs are publicly available at [30]. The repository includes RTL design files for the MAC, convolution and pooling layer, along with their corresponding testbench files, as well as Python implementation of the convolution and pooling layers in POSIT.

TABLE II
TOP-1, TOP-5 ACCURACY AND MATCH WITH FP32 FOR VARIOUS FORMATS

Format	**Accuracy (%)**		**FP32 Ref. (%)**	**Prediction Match with FP32 (%)**
	Top-1	**Top-5**		
POSIT(8,1)	90.5	99	90.8	92.4
POSIT(8,2)	89.6	99	90.8	96.9
POSIT(8,3)	82.0	99	90.8	89.5
FP16	90.7	99	90.8	99.7

V. CONCLUSION

This work optimizes convolution operations by reformulating them as matrix multiplications with DSP packing for efficient INT4 computation. A custom MAC unit, computing $a \times b + c \times d + e \times f$, supports posit formats (8,1), (8,2), and (8,3) for efficient DSP use. Applied to LeNet-5, our design shows a 0.3% accuracy drop compared to FP32, reducing LUT usage by 15% and demonstrating the Triple-Packed Posit MAC's effective balance of accuracy and hardware efficiency for resource-constrained deep neural network acceleration.

As future work, the proposed architecture can be extended to support larger and deeper CNN models such as VGG, ResNet, or MobileNet to evaluate scalability and performance under higher computational demands. Integrating the Triple-Packed Posit MAC into systolic array–based accelerators could further improve throughput and data reuse, making the design more suitable for high-performance inference. Additional exploration of adaptive posit configurations, mixed-precision strategies, and memory-aware optimizations may further enhance efficiency while preserving model accuracy.

REFERENCES

[1] J. Sommer, M. A. Özkan, O. Keszocze and J. Teich, "DSP-Packing: Squeezing Low-precision Arithmetic into FPGA DSP Blocks," 2022 32nd International Conference on Field-Programmable Logic and Applications (FPL), Belfast, United Kingdom, 2022, pp. 160-166.

[2] B. Ghavami, M. Sajadi, L. Shannon and S. Wilton, "Boosting Multiple Multipliers Packing on FPGA DSP Blocks via Truncation and Compensation-based Approximation," 2024 IEEE Computer Society Annual Symposium on VLSI (ISVLSI), Knoxville, TN, USA, 2024, pp. 222-227.

[3] X. Hu, X. Li, H. Huang, X. Zheng and X. Xiong, "TiNNA: A Tiny Accelerator for Neural Networks With Efficient DSP Optimization," in IEEE Transactions on Circuits and Systems II: Express Briefs, vol. 69, no. 4, pp. 2301-2305.

[4] P. J. Edavoor, A. Raveendran, D. Selvakumar, V. Desalphine, D. S. G and G. Raut, "Design and Analysis of Posit Quire Processing Engine for Neural Network Applications," 2023 36th International Conference on VLSI Design and 2023 22nd International Conference on Embedded Systems (VLSID), Hyderabad, India, 2023, pp. 252-257.

[5] J. Sun and Y. Lao, "Efficient Data Extraction Circuit for Posit Number System: LDD-Based Posit Decoder," in IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems, vol. 43, no. 6, pp. 1919-1923.

[6] S. S. Ram and K. Varghese, "Efficient Hardware Design of Parameterized Posit Multiplier and Posit Adder," 2023 IEEE Asia Pacific Conference on Circuits and Systems (APCCAS), Hyderabad, India, 2023, pp. 343-347.

[7] O. Kokane, M. Lokhande, G. Raut, A. Teman and S. K. Vishvakarma, "LPRE: Logarithmic Posit-enabled Reconfigurable edge-AI Engine," 2025 IEEE International Symposium on Circuits and Systems (ISCAS), London, United Kingdom, 2025, pp. 1-5.

[8] K. Elsaid, M. Safar and M. W. El-Kharashi, "Optimized FPGA Architecture for Machine Learning Applications using Posit Multipliers," 2022 International Conference on Microelectronics (ICM), Casablanca, Morocco, 2022, pp. 50-53.

[9] M. K. Jaiswal and H. K. . -H. So, "PACoGen: A Hardware Posit Arithmetic Core Generator," in IEEE Access, vol. 7, pp. 74586-74601.

[10] M. K. Jaiswal and H. K. . -H. So, "Universal number posit arithmetic generator on FPGA," 2018 Design, Automation Test in Europe Conference Exhibition (DATE), Dresden, Germany, 2018, pp. 1159-1162.

[11] S. Nambi, S. Ullah, S. S. Sahoo, A. Lohana, F. Merchant and A. Kumar, "ExPAN(N)D: Exploring Posits for Efficient Artificial Neural Network Design in FPGA-Based Systems," in IEEE Access, vol. 9, pp. 103691-103708, 2021.

[12] S. Walia, B. V. Tej, A. Kabra, J. Devnath and J. Mekie, "Fast and Low-Power Quantized Fixed Posit High-Accuracy DNN Implementation," in IEEE Transactions on Very Large Scale Integration (VLSI) Systems, vol. 30, no. 1, pp. 108-111, Jan. 2022

[13] M. H. Yacoub, S. M. Ismail and L. A. Said, "Hardware Realization of High-Speed Area-Efficient Floating Point Arithmetic Unit on FPGA," 2024 International Conference on Machine Intelligence and Smart Innovation (ICMISI), Alexandria, Egypt, 2024, pp. 190-193.

[14] A. Immaneni, S. Ullah, S. Nambi, S. S. Sahoo and A. Kumar, "PosAx-O: Exploring Operator-level Approximations for Posit Arithmetic in Embedded AI/ML," 2022 25th Euromicro Conference on Digital System Design (DSD), Maspalomas, Spain, 2022, pp. 214-223.

[15] Z. Carmichael, H. F. Langroudi, C. Khazanov, J. Lillie, J. L. Gustafson and D. Kudithipudi, "Deep Positron: A Deep Neural Network Using the Posit Number System," 2019 Design, Automation Test in Europe Conference Exhibition (DATE), Florence, Italy, 2019, pp. 1421-1426.

[16] J. L. Gustafson and I. T. Yonemoto, "Beating floating point at its own game: Posit arithmetic," Supercomput. Front. Innov., vol. 4, no. 2, pp. 71–86, 2017.

[17] "IEEE Standard for Floating-Point Arithmetic," in IEEE Std 754-2008 , vol., no., pp.1-70, 29 Aug. 2008.

[18] C. Leong, "Softposit," https://gitlab.com/cerlane/SoftPosit, 2018.

[19] R. Murillo, A. A. Del Barrio and G. Botella, "Customized Posit Adders and Multipliers using the FloPoCo Core Generator," 2020 IEEE International Symposium on Circuits and Systems (ISCAS), Seville, Spain, 2020, pp. 1-5.

[20] R. Murillo, D. Mallasén, A. A. Del Barrio and G. Botella, "Energy-Efficient MAC Units for Fused Posit Arithmetic," 2021 IEEE 39th International Conference on Computer Design (ICCD), Storrs, CT, USA, 2021, pp. 138-145

[21] L. Crespo, P. Tomás, N. Roma and N. Neves, "Unified Posit/IEEE-754 Vector MAC Unit for Transprecision Computing," in IEEE Transactions on Circuits and Systems II: Express Briefs, vol. 69, no. 5, pp. 2478-2482, May 2022.

[22] Z. Huang, S. Zhang and W. Wang, "An Efficient Method of Parallel Multiplication on a Single DSP Slice for Embedded FPGAs," in IEEE Access, vol. 7, pp. 100993-101008.

[23] Xilinx Inc., UltraScale Architecture DSP Slice User Guide, UG579, v1.11, July 2023. [Online]. Available: https://docs.amd.com/v/u/en-US/ug579-ultrascale-dsp

[24] S. Lee, D. Kim, D. Nguyen and J. Lee, "Double MAC on a DSP: Boosting the Performance of Convolutional Neural Networks on FPGAs," in IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems, vol. 38, no. 5, pp. 888-897.

[25] V. Chetlur et al., "cuDNN: Efficient primitives for deep learning," arXiv preprint arXiv:1410.0759, 2014.

[26] G. J. Hekstra and R. Nouta, "A Fast Parallel Multiplier Architecture," in Proc. IEEE Int. Symp. on Circuits and Systems (ISCAS), 1992, vol. 5, pp. 2128-2131.

[27] A. N. Danysh and E. E. Swartzlander, Jr., "A Recursive Fast Multiplier," in Proc. IEEE Asilomar Conf. Signals, Systems and Computers, 1998, pp. 197-201.

[28] sgpositpy GitHub repository. [Online]. Available: https://github.com/xman/sgpositpy

[29] J. Li, T. Li, G. Shen, D. Zhao, Q. Zhang and Y. Zeng, "Revealing Untapped DSP Optimization Potentials for FPGA-Based Systolic Matrix Engines," 2024 34th International Conference on Field-Programmable Logic and Applications (FPL), Torino, Italy, 2024, pp. 197-203

[30] "Triple-Packed POSIT MAC". Available: https://github.com/TeoZakeru/Triple-Packed-Posit-MAC.git